

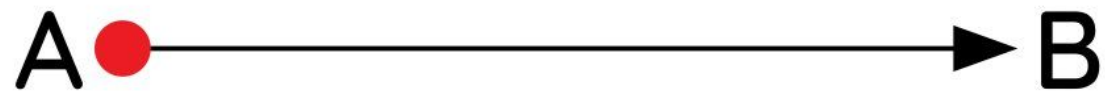
Efficient models: Introduction

Egor Shvetsov

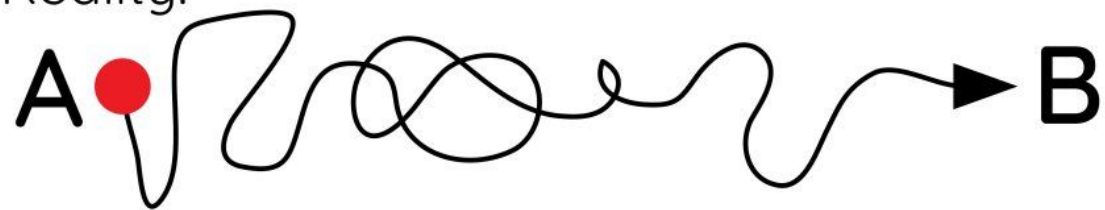
>>>

Expectations

Expectation:



Reality:



	Описание	Выдаем	Сдаем	Баллы
ДЗ-1	Профилировка и расчет сверток.	19/9/2026	26/9/2026	5
ДЗ-2	Воспроизводим и объясняем статьи.	28/9/2026	10/10/2026	5
Проекты	Групповой проект.	28/9/2026	05/12/2026	10
ДЗ-3	Задания на Triton.	12/10/2026	05/12/2026	10

Лекции, семинары и презентации прошлого года:

ИТМО 2025 - <https://github.com/On-Point-RND/Efficient-Models-course-ITMO-2025>

ШАД 2026 - <https://github.com/On-Point-RND/CEFM-2026-Computationally-Efficient-Models-YDS>

Project - 3 people MAX

Projects list (TBC)

Final Deliverable (due by 05/12):

- Full implementation of assigned tasks
- GitHub repository containing code, instructions, results description, and a final presentation (PDF)

Maximum score: **30 points**

- **18–20 points** → Pass
- **21–25 points** → Good
- **26–30 points** → Excellent

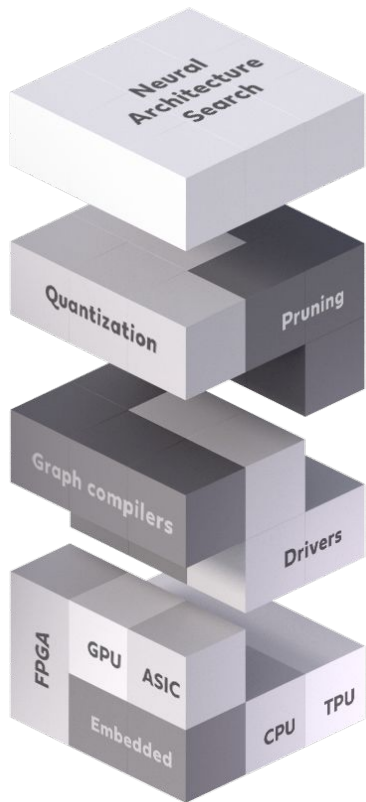


>>> What is future of AI?

YOUR THOUGHTS !?

- Everything is changing very fast and that the best skill is adaptability
- My goal of the course is that you got something useful from it
- Part of the course is about "techniques" but once we combine these techniques you will get a big picture of what is happening





HARDWARE



- GPU
- TPU
- CPU
- FPGA
- Ascend
- etc.

SOFTWARE



- CUDA
- Graph Compilers
- MKL - DNN
- etc.

ALGORITHMIC



- PRUNING
- QUANTIZATION
- NAS
- etc.

The future, future is uncertain but there are some certain constraints actually.

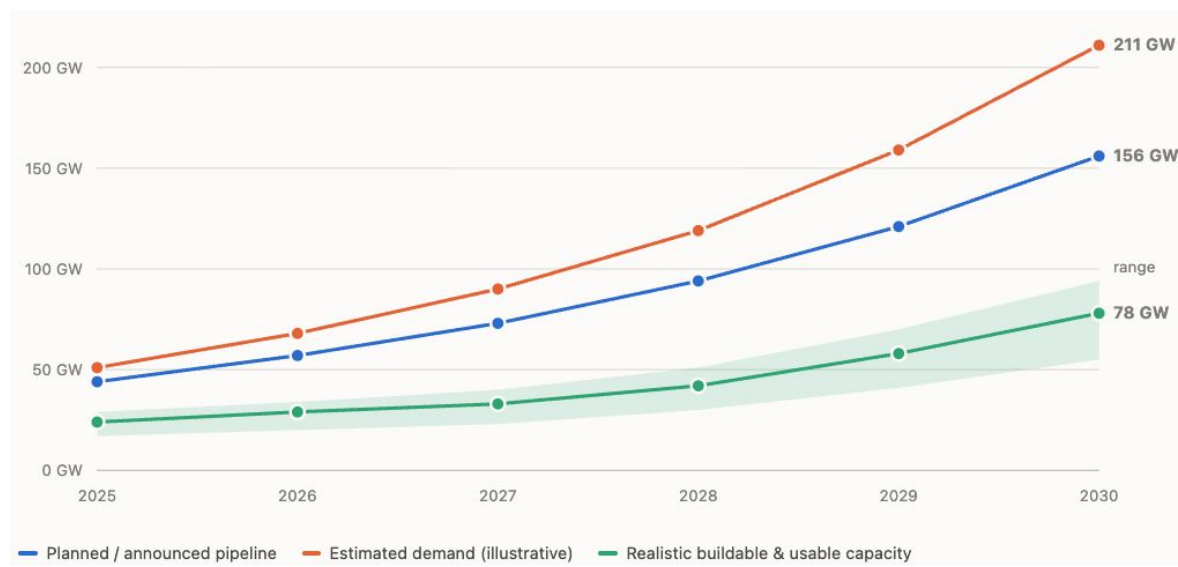


The future, future is uncertain but there are some certain constraints actually.

Computational efficiency and Energy for example, if we talk about agents for computational demand is huge and subscription barely covers it.

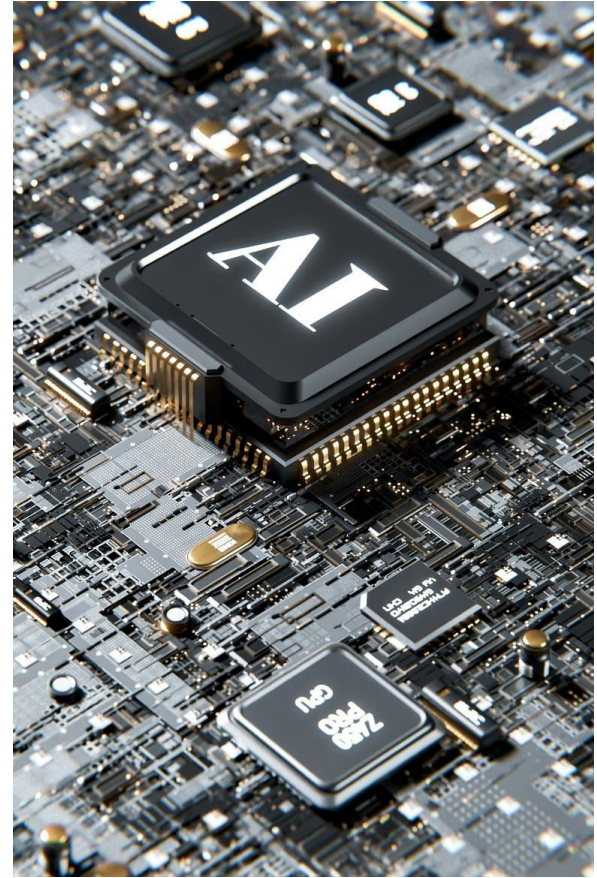
Building more power plants and producing cheap energy tomorrow is unrealistic - it takes years, so here we are.





- **Grid interconnection queues** - about a third of US data centers may need to self-generate power by 2030 simply because the grid connection isn't coming in time.
- **Equipment lead times** - gas turbines (cooling) and grid transformers are backlogged for years, independent of any single company's plans.
- **Water availability is regional, not national** - no nationwide shortage, but arid markets (parts of Arizona, drought-stressed California) already cap new development on potable-water limits.

The first home-project is to build an analytical model to predict FLOps, Memory, Latency and energy consumption of a model.



LLM usage on during the course

Do it yourself



Use LLMs



LLM usage on during the course

Do it yourself



Use LLMs



>>>

Motivations

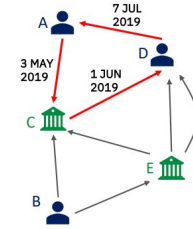
Dataset Example - How to model it?

Do we model it as:

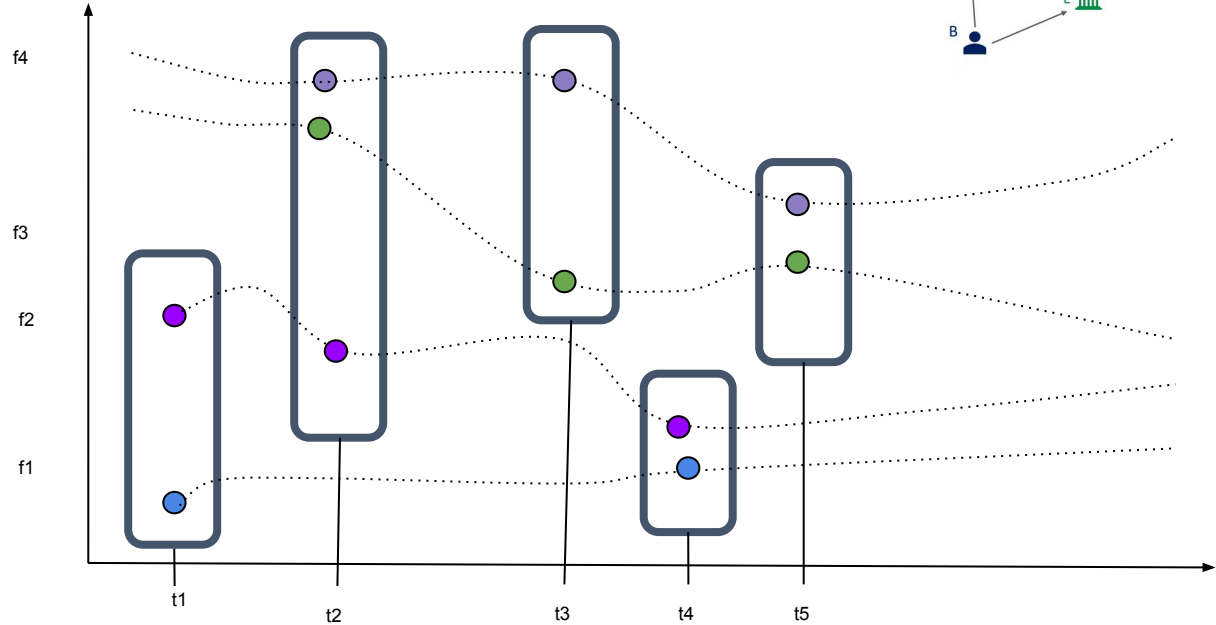
- Sequences
- Tabular data
- Graph

Does it depend on a problem?

Trans. ID	Timestamp	Source bank ID	Source Account	Target bank ID	Target Account	Amount	Currency	Payment type
0	3 MAY 2019 12:45	1	A	1	C	1400	USD	Cheque
1	15 MAY 2019 07:34	2	B	1	C	710	EUR	ACH
2	18 MAY 2019 16:55	3	E	1	C	950	USD	Credit card
3	1 JUN 2019 10:06	1	C	3	D	1200	CHF	Wire
4	27 JUN 2019 13:18	3	E	3	D	2300	EUR	Credit card
5	7 JUL 2019 11:14	3	D	1	A	1100	USD	Credit card
6	14 JUL 2019 09:37	2	B	3	E	650	USD	ACH
7	20 JUL 2019 14:02	3	E	3	D	2500	USD	Wire



- f4 Amount
- f3 Product category
- f2 Geo location
- f1 Currency



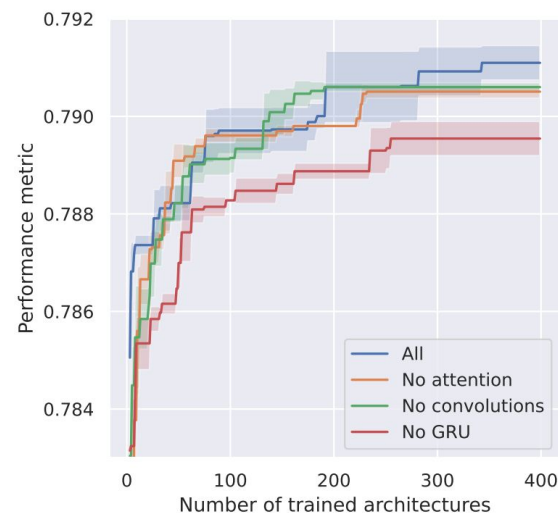
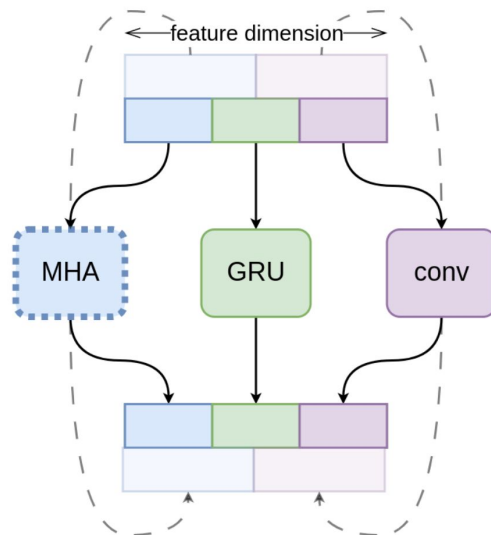
Real life example:

Encoder has both a searchable number of layers and the operations within each layer. Input of each Encoder layer

is divided into one to three blocks along the feature dimension, which can be processed using one of six potential

operations, such as MHA, GRU, or Convolution. The number of heads in MHA is searchable and is chosen from the set $\{1, 2, 4, 8\}$. In total it provides up to 19 variations for a single Encoder layer. It is worth noting that each layer has a distinct set of operations. The outputs from each block are concatenated and sent to the next layer within the

Encoder. This structure is illustrated in Figure 5. The encoder may have three possible values for the number of layers (1, 2, or 4), resulting in approximately 130×10^3 possible Encoder variations. [1]



Example:

Top-performing banks can potentially save between \$1-2\$ million USD per year for every 0.5% increase of classification performance, as measured by AUC, in some applied problems, involving classification of users' transactions.

Developing a new model and implementing it into a bank's system could cost approximately \$100,000 to \$150,000 over a 6-month project period, with half of the effort usually dedicated to model development.

How to find such models?
What efficiency depends on?

Example:

Top-performing banks can potentially save between \$1-2\$ million USD per year for every 0.5% increase of classification performance, as measured by AUC, in some applied problems, involving classification of users' transactions.

Developing a new model and implementing it into a bank's system could cost approximately \$100,000 to \$150,000 over a 6-month project period, with half of the effort usually dedicated to model development.

How to find such models? What efficiency depends on?

- Data pre-processing and transformations
- Expert domain knowledge
- Hyper-parameter optimization
- Architecture design
- Else ... (Your ideas) ?

But search space might be too big?



>>> Motivation type two
Compute

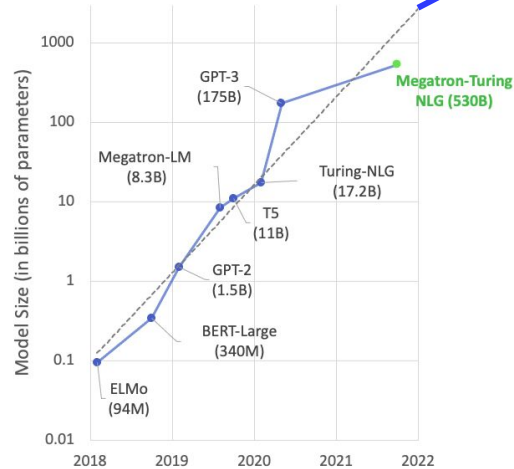
Energy and physical limitations

So what to do?

Edge devices
Autonomous devices
Robotics

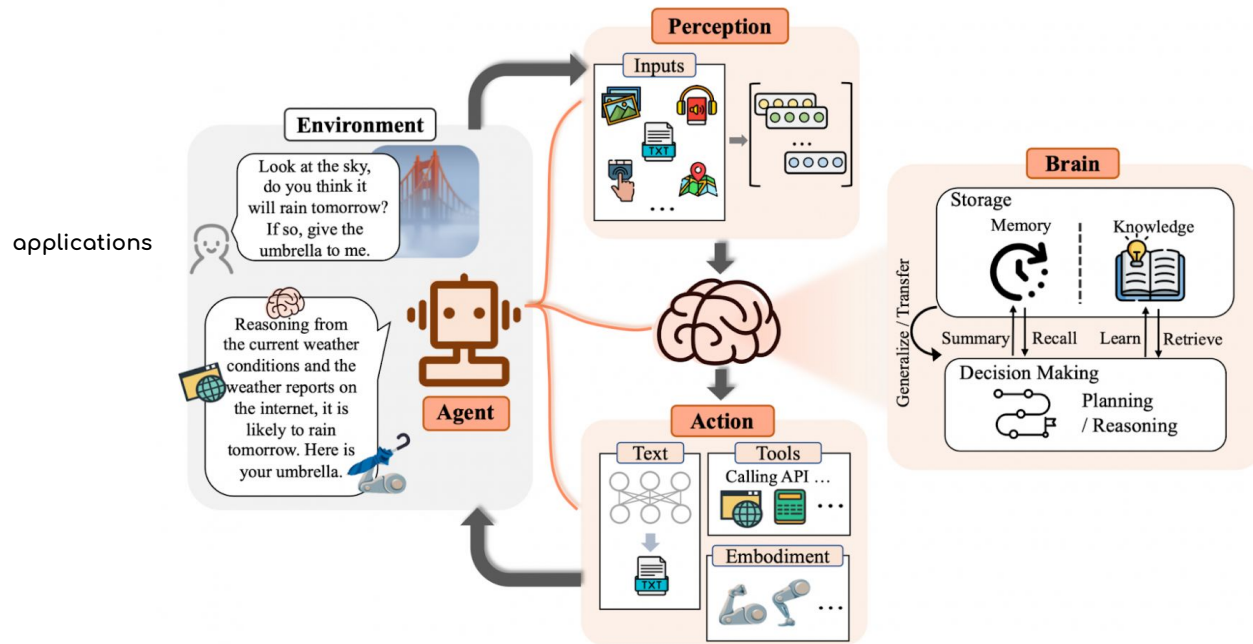


Modern LLMs



Trends

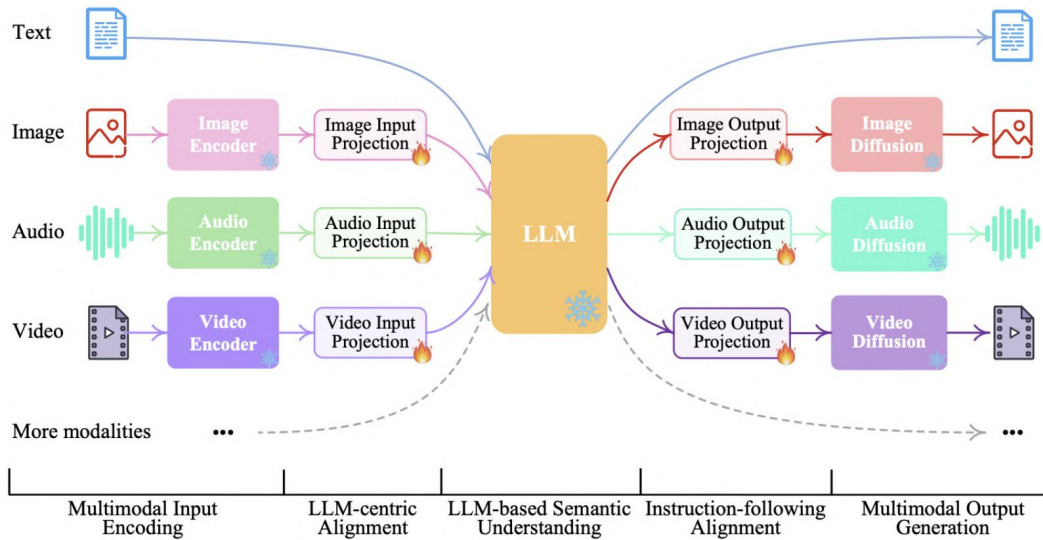
- LLM
- LLM based agents
- Multimodal models
- Scientific



Trends

- LLM
- LLM based agents
- Multimodal models
- Scientific

applications

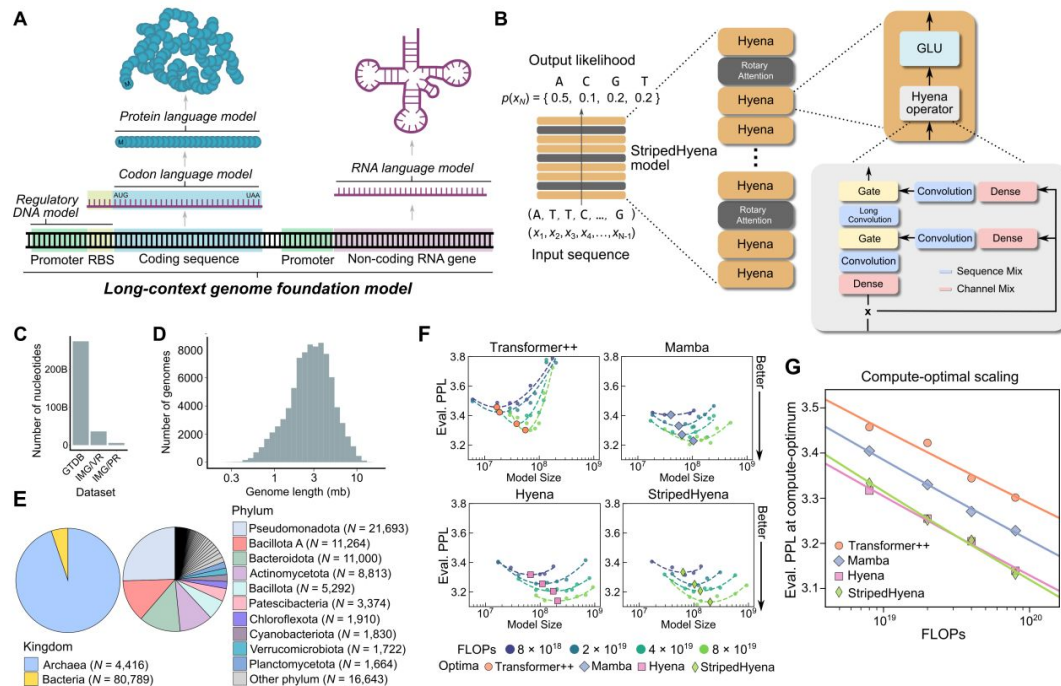


Trends

- LLM
- LLM based agents
- Multimodal models
- Scientific

applicat

Max sequence lengths: 131 000

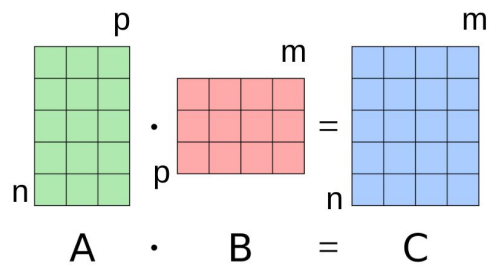


paper: [Sequence modeling and design from molecular to genome scale with Evo](#)

How do we measure
computational efficiency?

Memory
FLOPs
Latency
Energy

$$F[(n \times p) \times (p \times m)] = ?$$

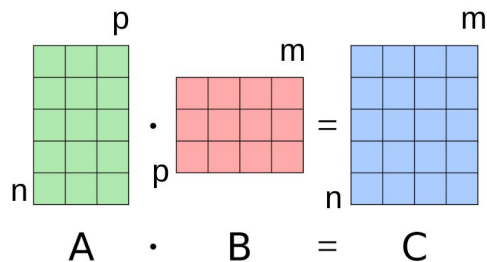


Memory
FLOPs
Latency
Energy

FLOPs - number of floating point operations?

How much is it for a matrix multiplication?

$$F[(n \times p) \times (p \times m)] = 2nmp$$

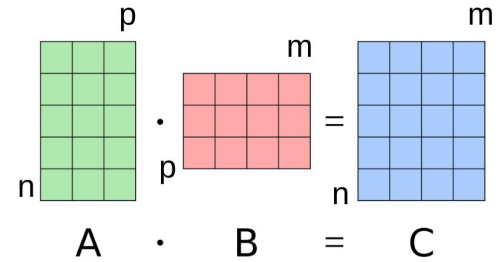


?

Multiply-accumulate (MAC) or multiply-add (MAD) operation is a common step that computes the product of two numbers and adds that product to an accumulator. The hardware unit that performs the operation is known as a multiplier-accumulator (MAC unit).

Сколько флопов уходит на умножение матриц?

- $2npr$ сложений и lmp умножений, всего $2npr$ флопов.
- Это 8 умножений для матриц размера 2×2 .
- Алгоритм Штрассена делает это за 7 умножений.
- На практике не используется из-за архитектуры «железа».
- На самом деле ответ pmp , а не $2pmp$ потому что [fma](#). (Fused multiply-add)



Memory
FLOPs
Latency
Energy

- Под скоростью работы часто понимают разные вещи.
- Latency — время отклика, то есть время от воздействия до реакции.
- Throughput — пропускная способность, то есть сколько данных можно обработать за единицу времени.

А в чем разница - примеры ?

Open router prices

Models

Search Compare

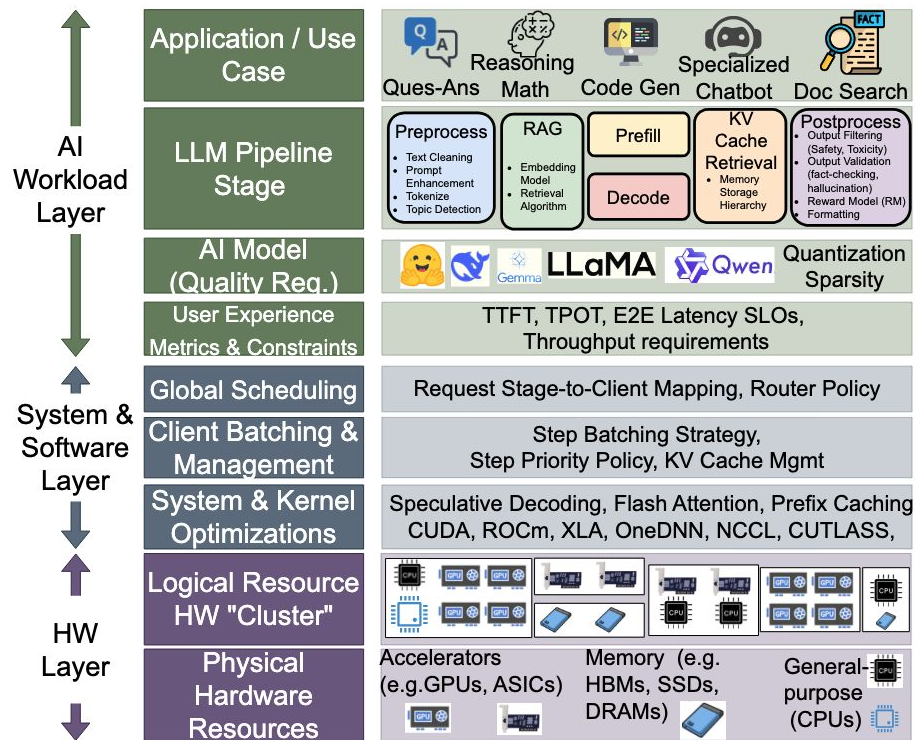
621 models

Latency: Low to High

Model Name	Weekly Tokens	Input (\$/1M)	Output (\$/1M)	Context
 NVIDIA: Nemotron Nano 12B 2 VL	69.9M	\$0.20	\$0.60	131,072
 Sao10K: Llama 3 8B Lunaris	6.36B	\$0.04	\$0.05	8,192 ⓘ
 Mistral: Mixtral 8x7B Instruct	1.76B	\$0.54	\$0.54	32,768
 NVIDIA: Llama 3.1 Nemotron Ultra 253B v1	2.32B	\$0.60	\$1.80	131,072
 Cohere: Command R7B (12-2024)	771M	\$0.0375	\$0.15	128,000
 Mistral: Mistral 7B Instruct	576M	\$0.20	\$0.20	32,768
 Meta: Llama Guard 4 12B	1.31B	\$0.18	\$0.18	163,840
 Mistral: Ministral 8B	1.4B	\$0.10	\$0.10	131,072
 Mistral: Mixtral 8x22B Instruct	156M	\$2	\$6	65,536
 Google: Gemma 3n 4B	946M	\$0.02	\$0.04	32,768
 OpenAI: gpt-oss-safeguard-20b	10.4B	\$0.075	\$0.30	131,072
 Cohere: Command R+ (08-2024)	23.6M	\$2.50	\$10	128,000
 NVIDIA: Llama 3.1 Nemotron 70B Instruct	75.6M	\$1.20	\$1.20	131,072
 Mistral: Ministral 3 3B 2512	1.07B	\$0.10	\$0.10	131,072
 Mistral: Devstral Small 1.1	1.61B	\$0.10	\$0.30	131,072
 Mistral: Voxtral Small 24B 2507	53.7M	\$0.10	\$0.30	32,000

Что влияет на цену кроме качества работы модели и ее размера?

Почему output дороже?



[source](#)

Что влияет на цену кроме качества работы модели и ее размера?

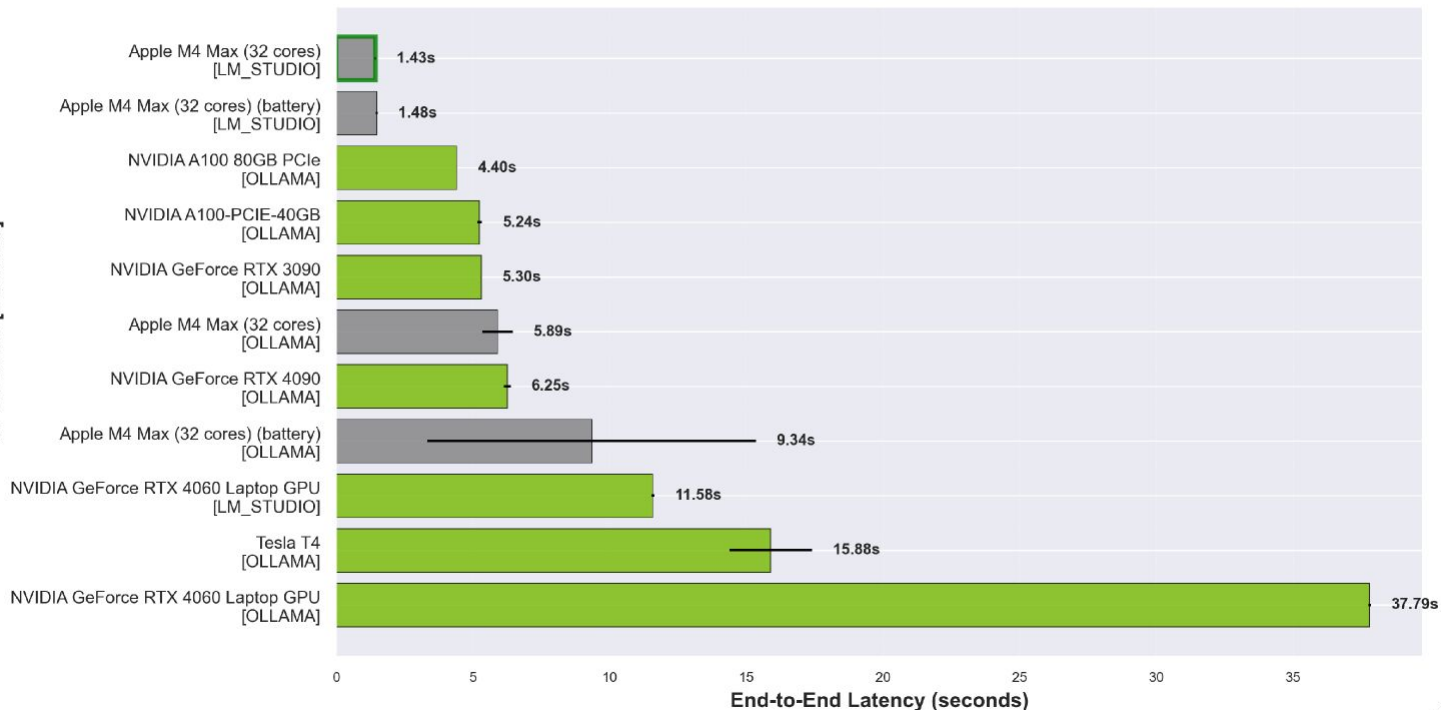
- Static batching
- Dynamic batching
- Continues batching (PagedAttention)
- KV-Cache strategy
- Flash attention version
- Prompt caching
- Speculative decoding
- Low - level kernel optimization
- Data type - FP16, Int8, Int4 ...



LLM Inference Performance: E2E Latency P50 (sec) [mean(P50) ± std(P50)] (Lower is Better)



GPU Device [Backend]



<https://github.com/bogdanminko/laperf>





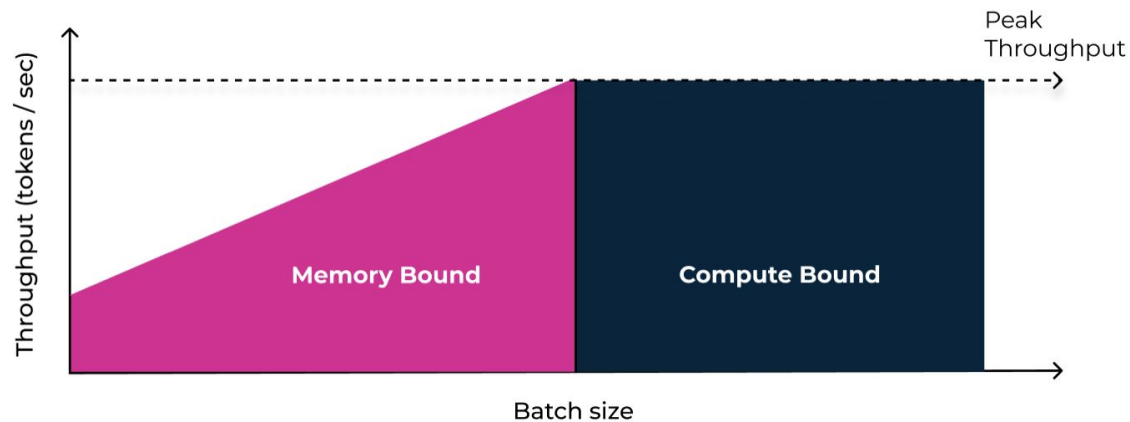
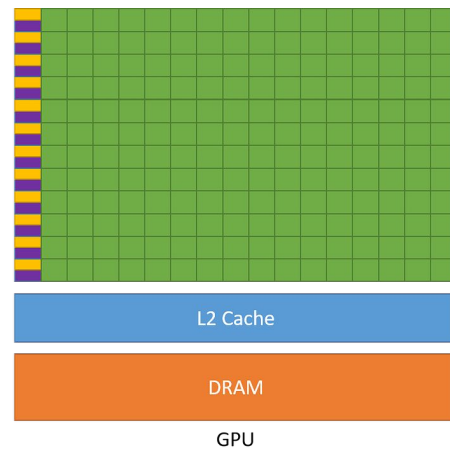
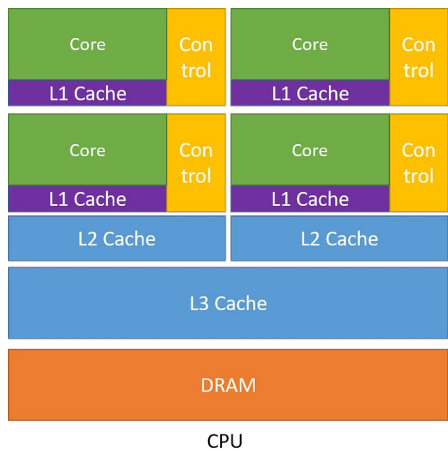
Latency optimized



Throughput optimized

Memory
FLOPs
Latency
Energy

Рекомендуем посмотреть: [Арифметика GPU](#)



Memory
FLOPs
Latency
Energy

An example:

Linear layers:

a weight matrix of M by M is used to process
a vector of M values with b bits.

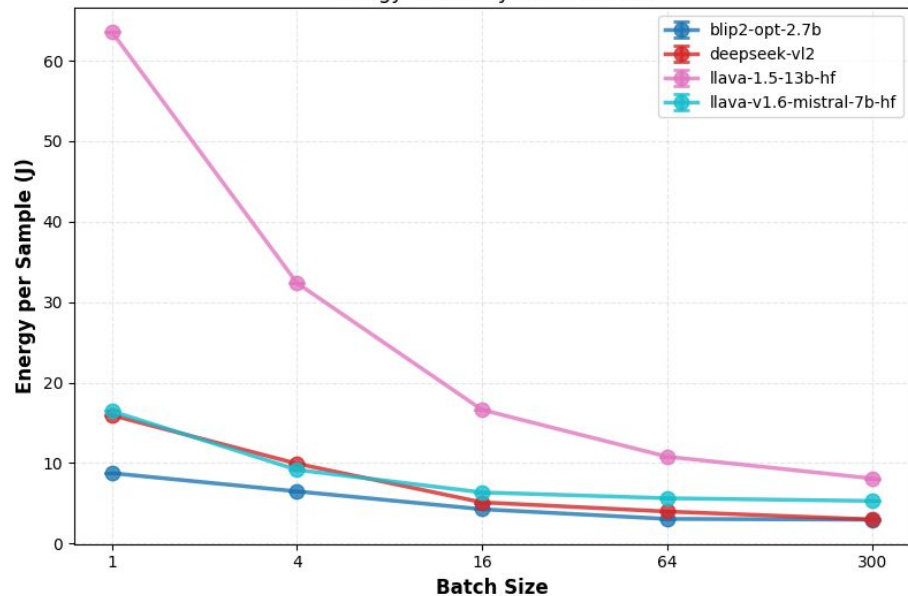
Total data transfer is: $b \times (M + M^2)$ or $\sim bM^2$

If the linear layer is used only for one vector M , it will require
to send the entire $\sim M^2$ matrix of weights as computation
occurs.

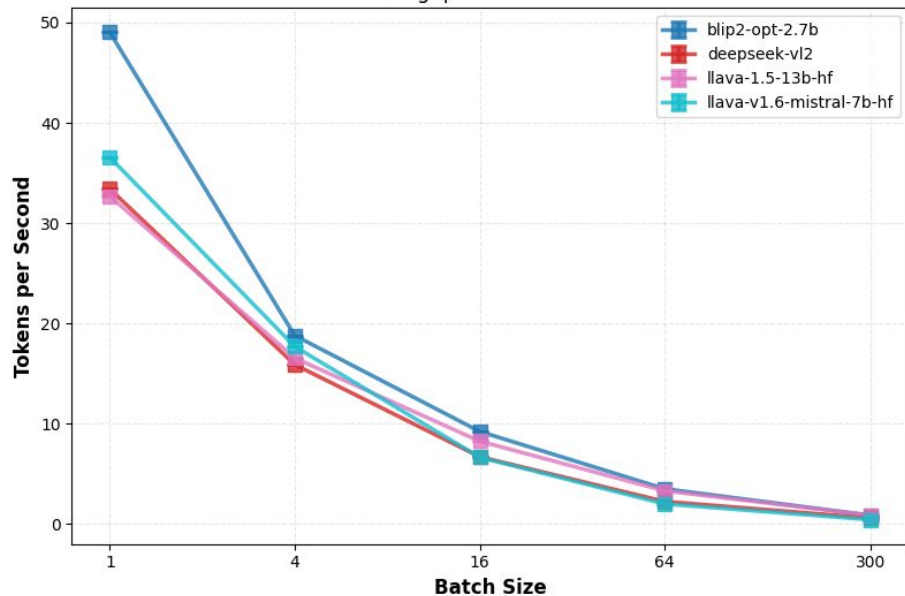
What happens if we have batch of vectors of size M ?

Energy Efficiency and Throughput vs Batch Size (COCO, system_prompt_100, 336x336)

Energy Efficiency vs Batch Size



Throughput vs Batch Size



[source](#)

Why energy per sample decreases?

Memory
FLOPs
Latency
Energy

Memory limited layers - слои которые почти не ограничены в арифметике:

- Ваши идеи.

Memory
FLOPs
Latency
Energy

Memory limited layers - слои которые почти не ограничены в арифметике:

- Normalization (scaling)
- Batch Normalization
- Activations
- Pooling
- Else ... (Your ideas) ?

How to understand if an operation is memory or arithmetic bound?

$$\text{Arithmetic Intensity} = \frac{\text{number of FLOPS}}{\text{number of byte accesses}} = \frac{2 \cdot (M \cdot N \cdot K)}{2 \cdot (M \cdot K + N \cdot K + M \cdot N)} = \frac{M \cdot N \cdot K}{M \cdot K + N \cdot K + M \cdot N}$$

Let's consider a $M \times N \times K = 8192 \times 128 \times 8192$ GEMM (General matrix multiply)

For this specific case, the arithmetic intensity is 124.1 FLOPS/B, lower than V100's 138.9 FLOPS/B, thus this operation **would be memory limited**.

Memory
FLOPs
Latency
Energy

Operation	Arithmetic Intensity	Usually limited by...
Linear layer (4096 outputs, 1024 inputs, batch size 512)	315 FLOPS/B	arithmetic
Linear layer (4096 outputs, 1024 inputs, batch size 1)	1 FLOPS/B	memory
Max pooling with 3x3 window and unit stride	2.25 FLOPS/B	memory
ReLU activation	0.25 FLOPS/B	memory
Layer normalization	< 10 FLOPS/B	memory

Memory
FLOPs
Latency
Energy

Input Type	Accumulation Type	Relative math throughput	Bandwidth savings
FP16	FP16	8x	2x
INT8	INT32	16x	4x
INT4	INT32	32x	8x
INT1	INT32	128x	32x

*Relative to FP32

Model	Params. (B)	Energy / Joule per response
Gemma 2 2B	2	40.42
Mistral 7B	7	43.76
Phi 3 Small	7	44.89
Llama 3.1 8B	8	51.12
Phi 3 Mini	4	54.59
Mistral Nemo	12	66.71
Gemma 2 9B	9	68.24
Phi 3 Medium	14	96.26
Mixtral 8x7B	47	121.49
Gemma 2 27B	27	192.55
Llama 3.1 70B	70	512.84
Mistral Large	123	869.17
Mixtral 8x22B	141	1161.61

How much energy consumes **Gemma 2 27B** in terms of 60W a light bulb:



- ~3 seconds of work
- ~5 seconds of work
- ~60 seconds on work

Model	Params. (B)	Energy / Joule per response
Gemma 2 2B	2	40.42
Mistral 7B	7	43.76
Phi 3 Small	7	44.89
Llama 3.1 8B	8	51.12
Phi 3 Mini	4	54.59
Mistral Nemo	12	66.71
Gemma 2 9B	9	68.24
Phi 3 Medium	14	96.26
Mixtral 8x7B	47	121.49
Gemma 2 27B	27	192.55
Llama 3.1 70B	70	512.84
Mistral Large	123	869.17
Mixtral 8x22B	141	1161.61

4000 mAh at 3.7V: $4000 \times 3.7 / 1000 = 14.8 \text{ Wh} \approx 15 \text{ Watt - hours}$

$14.8 \text{ Wh} \times 3600 \text{ J/Wh} = 53,280 \text{ J}$



One fully charged Phone:

~ 50 Mistral 8x22B requests

~ 1000 LLama 3.1 8B requests

Model	Params. (B)	Energy / Joule per response
Gemma 2 2B	2	40.42
Mistral 7B	7	43.76
Phi 3 Small	7	44.89
Llama 3.1 8B	8	51.12
Phi 3 Mini	4	54.59
Mistral Nemo	12	66.71
Gemma 2 9B	9	68.24
Phi 3 Medium	14	96.26
Mixtral 8x7B	47	121.49
Gemma 2 27B	27	192.55
Llama 3.1 70B	70	512.84
Mistral Large	123	869.17
Mixtral 8x22B	141	1161.61

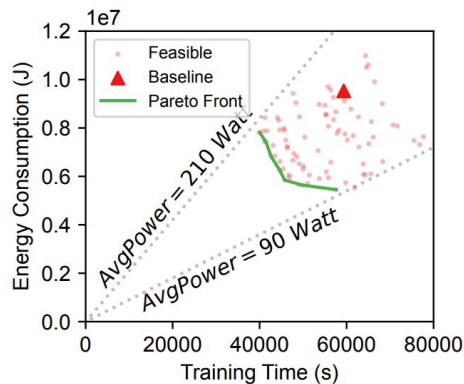
Brain consumes 20 joules energy
per second.



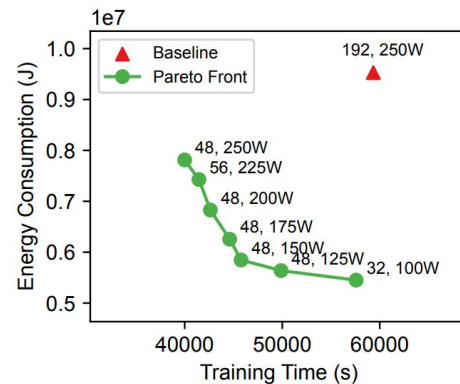
Memory
FLOPs
Latency
Energy

Some facts:

- AlphaGo: 1920 CPUs and 280 GPUs, \$3000 electric bill per game
- At extreme scales, training the GPT-3 model just once consumes **1,287 MWh** which is enough to supply an average US household for **120 years**.



(a) Energy-Time Tradeoff



(b) Pareto Front Zoom-in

Memory
FLOPs
Latency
Energy

Accessing data from DRAM can consume 100–1000x more energy than performing an arithmetic operation

[\[LINK\]](#)

ARITHMETIC OPERATION	ADD	MUL
8-BIT INTEGER	0.03 pJ	0.2 pJ
16-BIT FLOATING POINT	0.4 pJ	1.1 pJ
32-BIT INTEGER	0.1 pJ	3.1 pJ
32-BIT FLOATING POINT	0.9 pJ	3.7 pJ

MEMORY SIZE	64-BIT MEMORY ACCESS
8KB	10 pJ
32KB	20 pJ
1 MB	100 pJ
DRAM	1.3-2.6 nJ

>>>

Экономика LLM

Мы хотим более хорошие модели, для этого нам нужно:

- Больше данных
- Более большие модели

Более большие модели с большим числом данных будут обучаться дольше, но если обучать долго, модели могут устареть. Мало кто обучает модели дольше чем **пол года**.

Что делать?

Первый Трансформер 2017 обучался на 8 GPU и 12 часов.

GPT-3 (2018) 1024 GPU V100 и 34 дня

Llama 3.1 405B - 16 000 GPU и примерно 2 месяцев

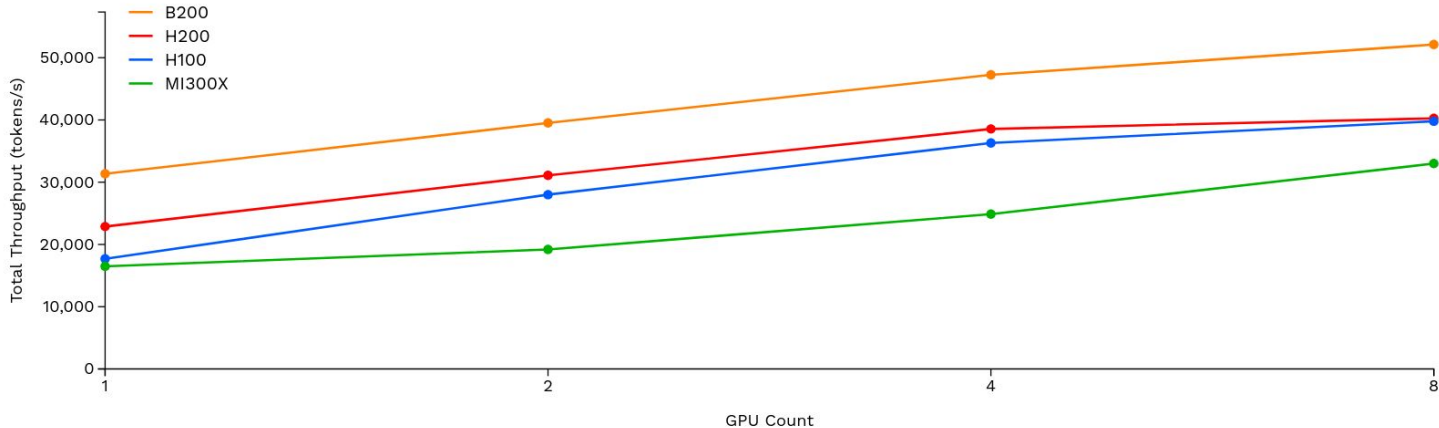


Initial scaling: Performance scales nearly linearly for small numbers of GPUs (e.g., 2 to 8), especially within a single node connected by high-speed links like NVLink or a fast PCI Express bus.

Bottlenecks and saturation: Scaling efficiency decreases as the number of GPUs increases, particularly when scaling across multiple nodes over an InfiniBand or Ethernet network. The all-reduce communication operation becomes the main bottleneck, as the time spent exchanging gradients grows with the number of GPUs.

Multi-GPU benchmark results

Total throughput vs. GPU count



[source](#)



How to scale to 1000 of
GPUs then?



How to scale to 1000 of
GPUs then?



Hybrid strategies:

(model parallels) x (data parallel) x (and more)

*Not the scope of this class!

Стоимость обучения

Увеличение
в 3,5 раза/год с 2020 года

Доверительный интервал 90%: от 2,8х до 4,4х раз.

Вычислительные ресурсы для обучения

Увеличение
в 5 раз/год с 2020 года.

Доверительный интервал 90%: от 2,8х до 4,4х раз.

Стоимость приобретения оборудования

Увеличение в 2,5 раза/год с 2016

Доверительный интервал 90%: от 2,1х до 2,9х раз.

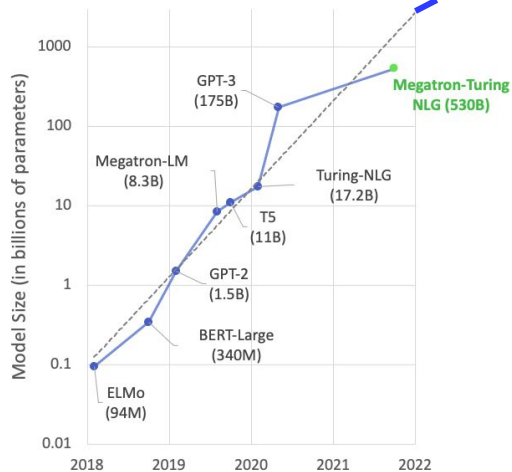
Пропускная способность памяти (DRAM)

Увеличение в 1,18 раза/год

Energy and physical limitations

again!

Modern LLMs



Пропускная способность памяти (DRAM)

Увеличение в 1,18 раза/год

Размер датасетов для обучения LLM

Увеличение в 3,6 раза/год с 2010

Когда закончатся данные?

В 2028

Согласно медианному-прогнозу, большая часть эффективного запаса общедоступного текста, созданного человеком, будет использована в 2028

Объем текстовых данных в интернете

510 триллионов токенов

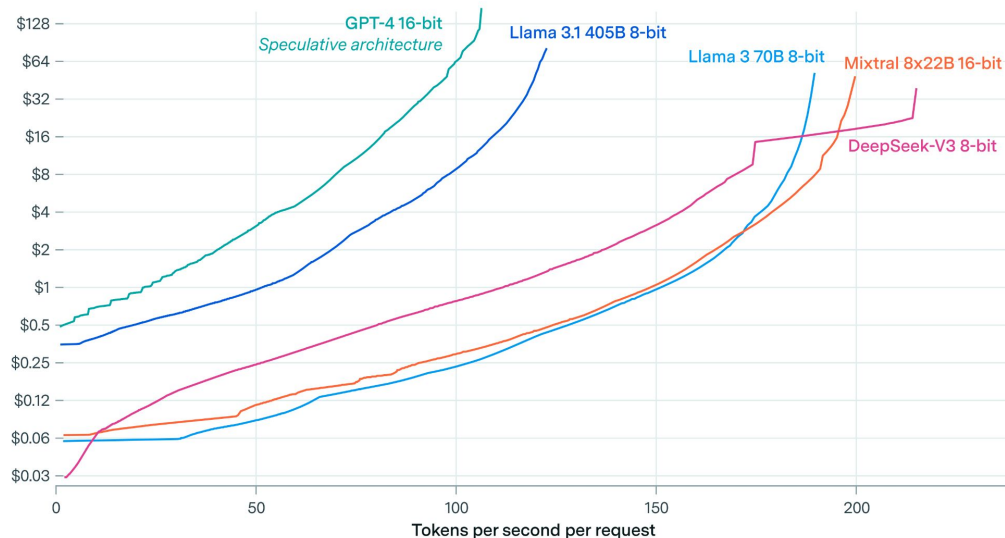
Доверительный интервал 95%: от 130 до 2100 триллионов токенов.

Token economics of various models

EPOCH AI

All models are assumed to run on H100 SXM GPUs costing \$2 per hour per GPU. We also assume speculative decoding is used in all cases with a decoder model matching Llama 3 8B's architecture and with a token acceptance probability of 80%.

Cost per million tokens generated (USD)



CC-BY

epoch.ai

Из чего формируется стоимость работы моделей.

Время вычислений — это время, которое занимают у ядер GPU выполнение основных операций: сложения и умножения.

Время чтения-записи в память — это время, необходимое для загрузки данных из высокоскоростной памяти (HBM) в ядра. В более сложных моделях учитывается вся иерархия памяти, но здесь мы это опускаем.

Время передачи по сети — вычисляется делением объема данных, которые получает каждый GPU, на пропускную способность его сетевого канала.

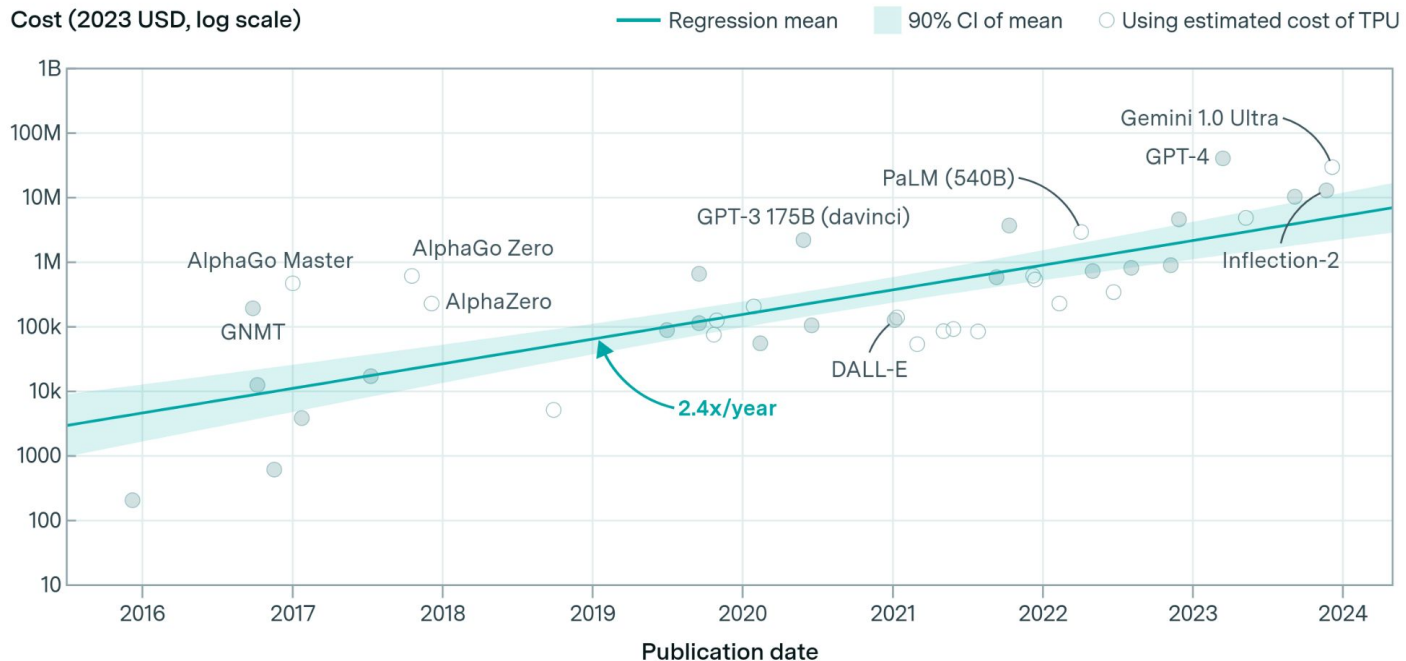
Задержка — это фиксированное время, которое занимают такие операции, как запуск задач и операции GPU, независимо от их размера.

(Например, протокол LL128 в библиотеке NCCL для операции *all-reduce* имеет базовую задержку в 30 микросекунд на машине DGX H100 — даже если передается всего один байт данных.)

Внимание, в этом расчете нет людей!

Сколько стоит обучить модель?

Amortized hardware and energy cost to train frontier AI models over time 



Сколько стоит обучить модель?

Breakdown of costs for training and experiments

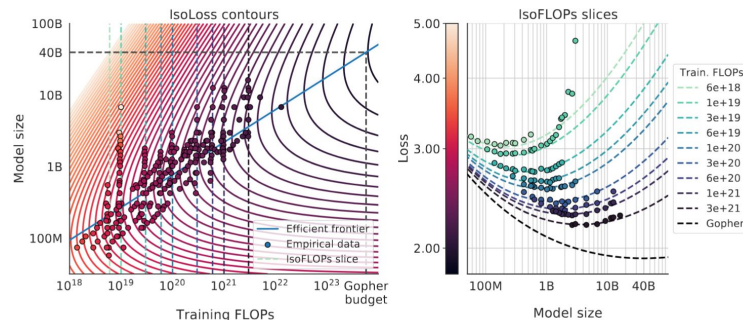


Scaling Laws

$$L(N, D) = \left(\frac{N_0}{N}\right)^{\alpha_N} + \left(\frac{D_0}{D}\right)^{\alpha_D}$$

Chinchilla's Golden Rule: Optimal tokens = 20 × model size

Model	Size (# Parameters)	Training Tokens
LaMDA (Thoppilan et al., 2022)	137 Billion	168 Billion
GPT-3 (Brown et al., 2020)	175 Billion	300 Billion
Jurassic (Lieber et al., 2021)	178 Billion	300 Billion
Gopher (Rae et al., 2021)	280 Billion	300 Billion
MT-NLG 530B (Smith et al., 2022)	530 Billion	270 Billion
Chinchilla	70 Billion	1.4 Trillion



Scaling Laws

$$L(N, D) = \left(\frac{N_0}{N}\right)^{\alpha_N} + \left(\frac{D_0}{D}\right)^{\alpha_D}$$

$$\alpha_N \approx 0.074, \quad \alpha_D \approx 0.11$$

Lever	Scaling Exponent	Real-World Impact
Model Size (N)	$\alpha_N = 0.076$	$\uparrow 10\times \text{ params} \rightarrow \downarrow 7.6\% \text{ loss " (e.g. 7B} \rightarrow 70\text{B params) "}$
Training Data (D)	$\alpha_D = 0.11$	$\uparrow 10\times \text{ tokens} \rightarrow \downarrow 11\% \text{ loss " (e.g. 300B} \rightarrow 3\text{T tokens) "}$
Compute (C)	$\gamma = 0.052$	$\uparrow 10\times \text{ FLOPs} \rightarrow \downarrow 5.2\% \text{ loss " (e.g. } 1\text{e}^{22} \rightarrow 1\text{e}^{23} \text{ FLOPs) "}$

Scaling Laws

$$L(N, D) = \left(\frac{N_0}{N}\right)^{\alpha_N} + \left(\frac{D_0}{D}\right)^{\alpha_D}$$

$$N \sim C^{0.7}, \quad D \sim C^{0.3}$$



Model	Parameters	Tokens	Loss (C4)	Inference Cost*
Chinchilla	70.0B	1.40T	1.45	\$0.0001/token
Gopher	280.0B	300.0B	1.58	\$0.0007/token
GPT-3	175.0B	300.0B	1.62	\$0.0005/token

Scaling Laws

$$L(N, D, C) = \frac{a}{(N \cdot \text{eff}(C))^b} + \frac{c}{D^d} + e.$$

Results:

eff(Q):

8-bit: 0.875

4-bit: 0.747

2-bit: 0.289

1-bit: 0.0067

eff(Sparse):

50%: 0.871

75%: 0.622

[Compression Scaling Laws: Unifying Sparsity and Quantization](#)

Scaling Laws

$$L(N, D, C) = \frac{a}{(N \cdot \text{eff}(C))^b} + \frac{c}{D^d} + e.$$

Results:

eff(Q):

8-bit: 0.875

4-bit: 0.747

2-bit: 0.289

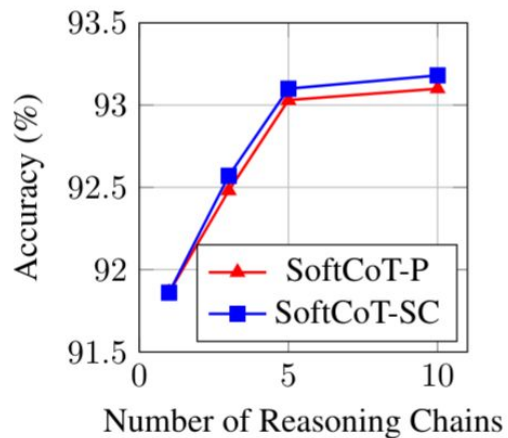
1-bit: 0.0067

eff(Sparse):

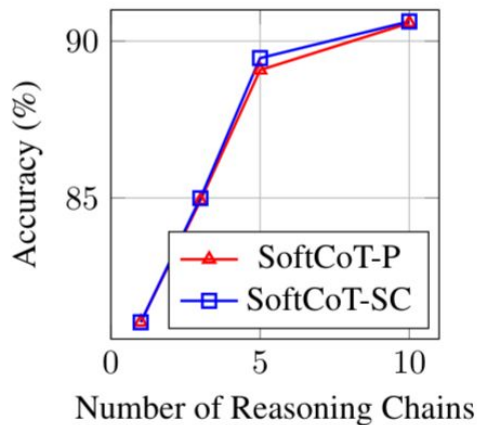
50%: 0.871

75%: 0.622

Scaling Laws



(a) Qwen3-8B



(b) LLaMA-3.1-8B-Instruct

TEST - TIME COMPUTE
SCALING



SoftCoT++: Test-Time Scaling with Soft Chain-of-Thought Reasoning
Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters

Scaling Laws

CONCERNS About
scaling laws?

CONCERN



Scaling Laws

Perplexity Cannot Always Tell Right from Wrong

Petar Veličković¹ Federico Barbero¹ Christos Perivolaropoulos¹ Simon Osindero¹ Razvan Pascanu¹

Abstract

Perplexity—a function measuring a model’s overall level of “surprise” when encountering a particular output—has gained significant traction in recent years, both as a loss function and as a simple-to-compute metric of model quality. Prior studies have pointed out several limitations of perplexity, often from an empirical manner. Here we leverage recent results on Transformer continuity to show in a rigorous manner how perplexity may be an unsuitable metric for model selection. Specifically, we prove that, if there is *any* sequence that a compact decoder-only Transformer model predicts accurately and confidently—a necessary pre-requisite for strong generalisation—it must imply existence of another sequence with very low perplexity, but not predicted correctly by that same model. Further, by analytically studying iso-perplexity plots, we find that perplexity will not always select for the more accurate model—rather, any increase in model confidence must be accompanied by a commensurate rise in accuracy for the new model to be selected.

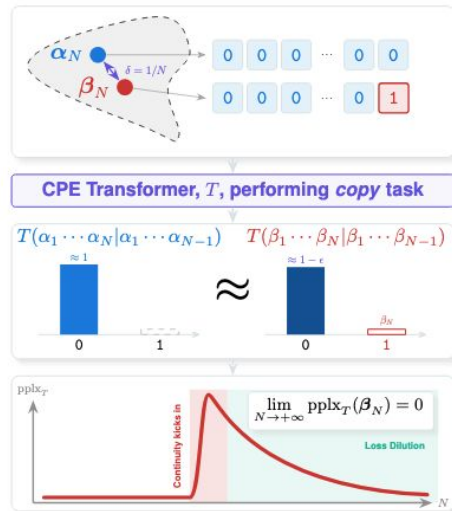


Figure 1. Using the continuity result of Pasten et al. (2025), we show that, if a (compact) Transformer T is confident in copying *any* long enough sequence α_N , then there must exist β_N which T fails to copy, yet, log-perplexity will tend to zero as N grows.

[source](#)

MODEL Compression: Why does it work & why can't we train a smaller model from the beginning?

MODEL Compression: Why does it work & why can't we train a smaller model from the beginning?

**Models are overparameterized
But why?**

Initialization

Training (optimization) dynamic

Data size and its quality

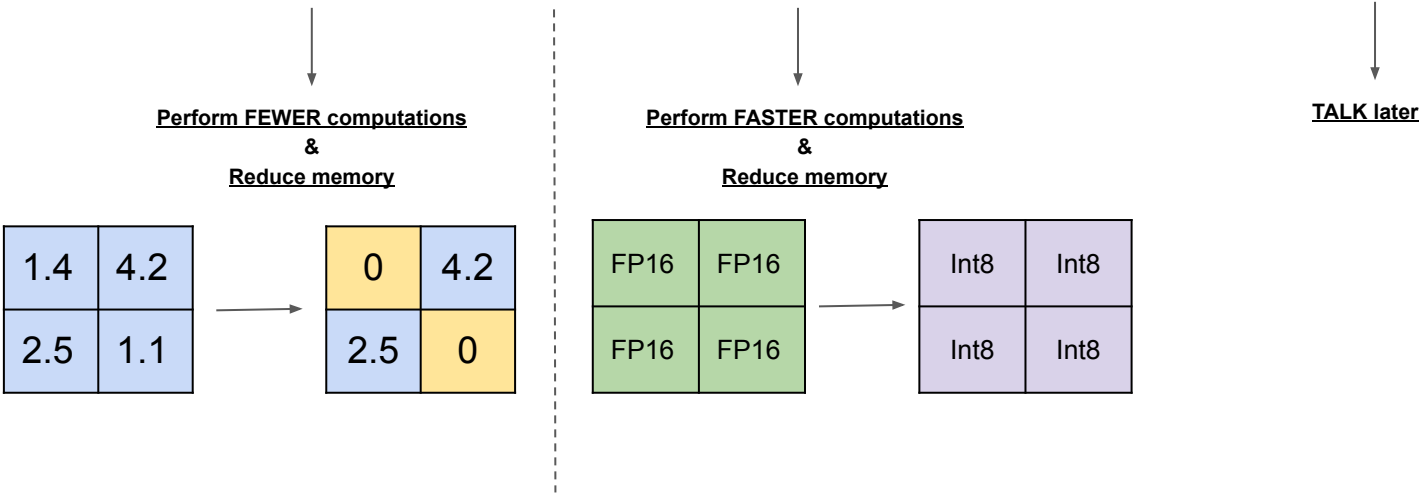
How to make models faster?

Reduce memory
Reduce FLOPS
Make operations faster

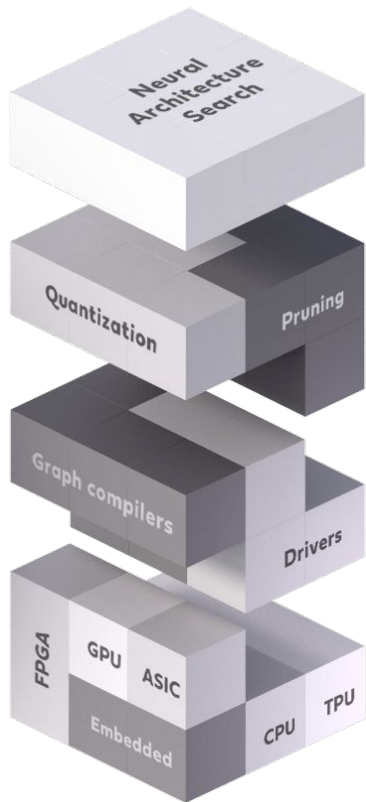
HOW?

Here we focus on resource efficient methods for faster NN training which accelerate training without significant usage of additional resources.

	General methods		Task or Architecture specific
Methods	Sparsification	Quantization	Heuristics
How	Remove some weights to <u>perform fewer computations</u> and reduce memory.	Change data type into a lower precision (e.g. FP16 -> Int8) to <u>perform faster</u> computations and reduce memory.	Find some flaws in the current design and fix them. E.g. token pruning in LLMs may not apply for diffusion based LLMs. Another approach is to use a cleaner dataset but this would be task specific.
Cons	Requires some structure or very high sparsity more than 90%. Otherwise Index operations would overcome all of the gains.	Coarse approximation and limited representative range of values. Requires both matrices e.g. X.TW to be the same data type for faster arithmetic.	Will not work for other tasks or architectures.
Pros	Preserves Floating Point range	Retains better performance for weight quantization than weight sparsification. Wider hardware support.	Sometimes leads to really good results.
	Requires hardware support and low level kernel implementation.		Vary a lot based on a task



Software meets Hardware



HARDWARE

- GPU
- TPU
- CPU
- FPGA
- Ascend
- etc.

SOFTWARE

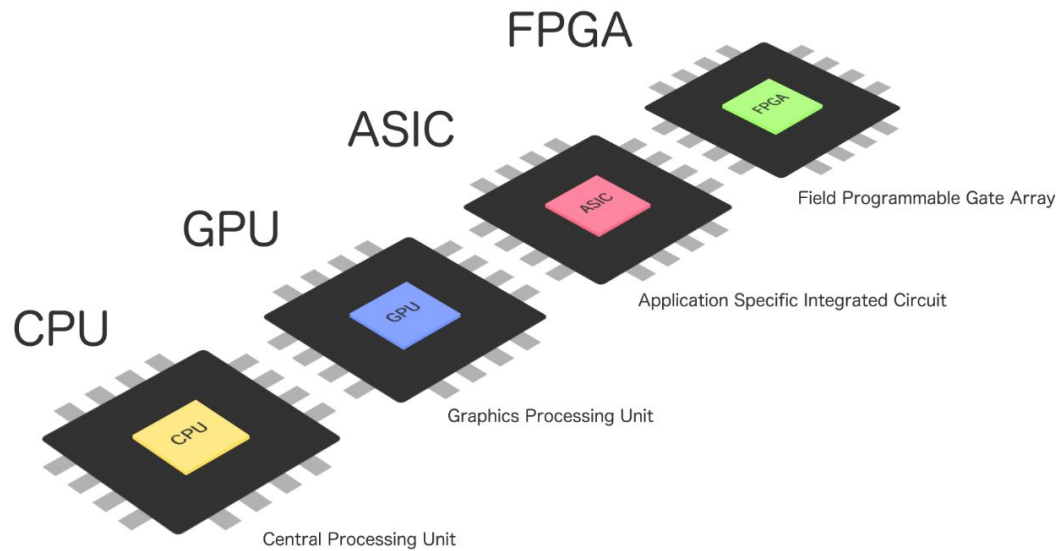
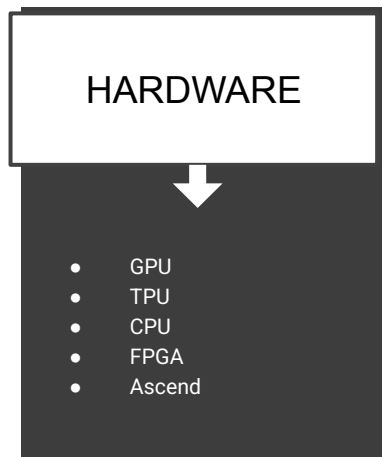
- CUDA
- Graph Compilers
- MKL - DNN
- etc.

ALGORITHMIC

- PRUNING
- QUANTIZATION
- NAS
- etc.

A majority of machine learning architectures lack explicit consideration for software or hardware properties during development, resulting in computational inefficiency.

Furthermore, many models concentrate exclusively on achieving optimal results rather than accounting for practical implementation issues.



CPU	The central processing unit (CPU) is a general-purpose processor typically containing between 4 and 16 cores.
GPU	Graphics Processing Units (GPUs) are highly parallel cores (100 or 1000) for high-speed graphics rendering. They provide high-performance processing and typically take up more space and consume more power than CPUs. Because of their large number of small cores, GPUs are well suited for AI workloads, facilitating both neural network training and AI operation.
FPGA	Field Programmable Gate Array (FPGA), a technology that creates a chip with a set of logic elements, flip-flops, sometimes RAM and programmable electrical connections between them, consumes less power than CPUs and GPUs. Can be reprogrammed on site by engineers with programming experience.
ASICs	Application-specific integrated circuits (ASICs) are custom logic designed using manufacturer circuit libraries and offer the benefits of low power consumption, speed and small footprint. However, they are time-consuming to develop and more expensive than other options, so ASICs are recommended for products that will operate in very high volumes. ASICs Types: - Image processing units (VPUs), image and image processors, and coprocessors - Tensor processing units (TPUs), such as the first TPU developed by Google for its machine learning platform, TensorFlow. - Neural Compute Units (NCU), including from ARM
QPU	A quantum processing unit (QPU) is the central component of a quantum computer or quantum simulator. It is a physical or simulated processor that contains a series of interconnected qubits that can be manipulated to compute quantum algorithms. A QPU uses the behavior of particles such as electrons or photons to perform certain types of calculations much faster than the processors in modern computers. QPUs rely on behavior called superposition, the ability of a particle to be in many states at once, described in a relatively new branch of physics called quantum mechanics. In contrast, CPUs, GPUs, and DPUs apply the principles of classical physics to electrical currents. This is why modern systems are called classical computers.
Neuromorphic chips	Neuromorphic computing—brain-inspired computing—has emerged as a new technology that can process information at very low energy costs using electronic devices that mimic the electrical behavior of (biological) neural networks.
Photonic computing	Оптические вычисления или фотонные вычисления используют световые волны, создаваемые лазерами или некогерентными источниками, для обработки данных, хранения данных или передачи данных для вычислений. На протяжении десятилетий фотоны обещают обеспечить более высокую пропускную способность, чем электроны, используемые в обычных компьютерах (см. оптические волокна).

Support of low precision computations in GPU

- Float16
- Float32
- Bfloat16

Supported by hardware and
software for most modern GPU

Not supported by
hardware or software

Will be discussed in future series ...



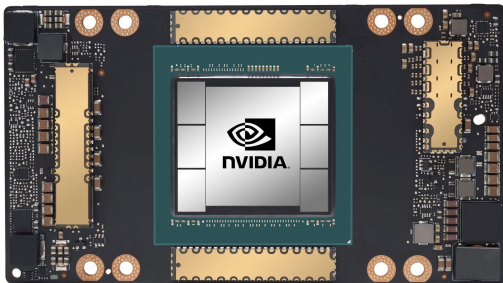
FP 32

FP 16

8 bit

4 bit

1 bit



Supported by some
hardware only

Cerebras Systems

California-based Cerebras Systems has unveiled the Wafer Scale Engine (WSE-3), its latest artificial intelligence (AI) chip with a whopping four trillion transistors. It delivers twice the performance of its predecessor, the Cerebras WSE-2, which previously held the record for the fastest chip.

Systems built using WSE-3 will be able to accurately train models with 70 billion parameters in just one day.

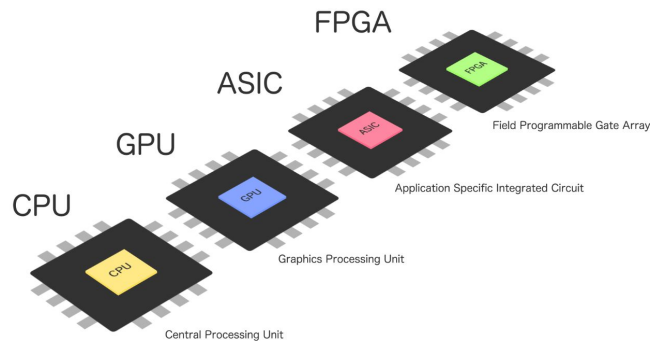
The chip is almost size of iPad.

Fast but still has a huge energy consumption.



FPGA

- Amazon has FPGA F1 instances in the cloud
- Alibaba has FGPA F3 instances in the cloud
- Yandex uses FPGA for its own DL output
- Microsoft launched (in 2015) Project Catapult, which uses FPGA clusters
- Microsoft Project Brainwave: FPGA-Based AI Inference
- Microsoft Azure allows you to deploy pre-trained models on FPGA(!)
- Baidu has FPGA instances.



ASIC

An ASIC (Application Specific Integrated Circuit) is an integrated circuit that is customized for a specific use rather than intended for general use.

- Google has Tensor Processing Units (TPU v2/v3/v4) in the cloud
- Intel has acquired Habana, Mobileye, Movidius, Nervana and has processors for training and inference
- Graphcore has a second generation IPU
- AWS has its own features for training and inference with ASICS
- Alibaba Hanguang 800
- Huawei Ascend 310, 910
- Bitmain Sophon, Cerebras, Groq and many, many others
- Many ASICs are designed for multi-chip and supercomputer configurations!

ASIC/FPGA

Problems with FPGA/ASIC and edge devices

Energy efficiency:

- Better than CPU/GPU, but still far from 20 watts used by the human brain

Architecture:

- Even more specialized for ML/DL computations

Neuromorphic computing

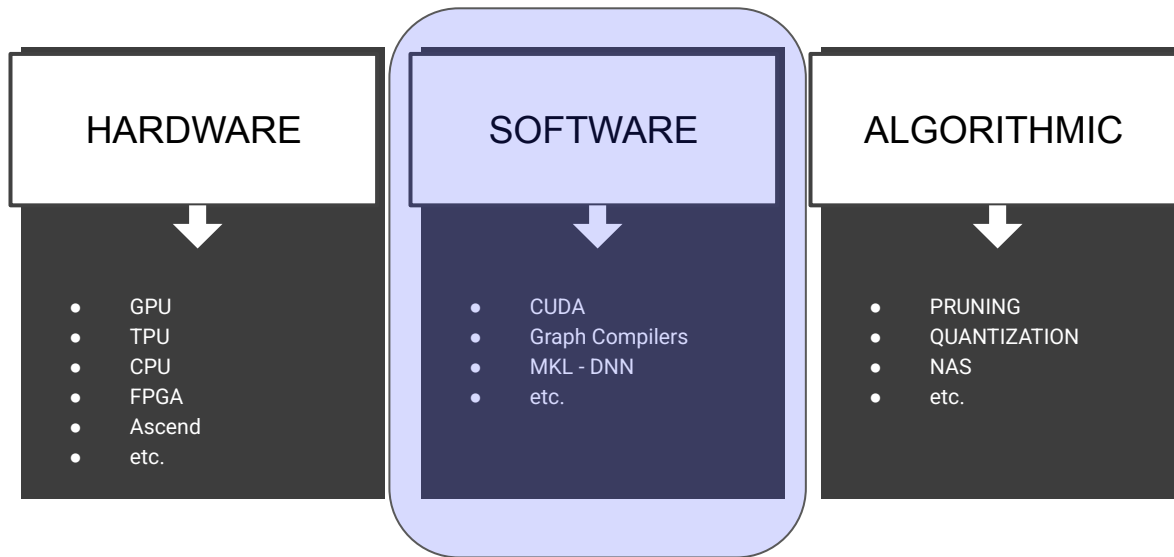
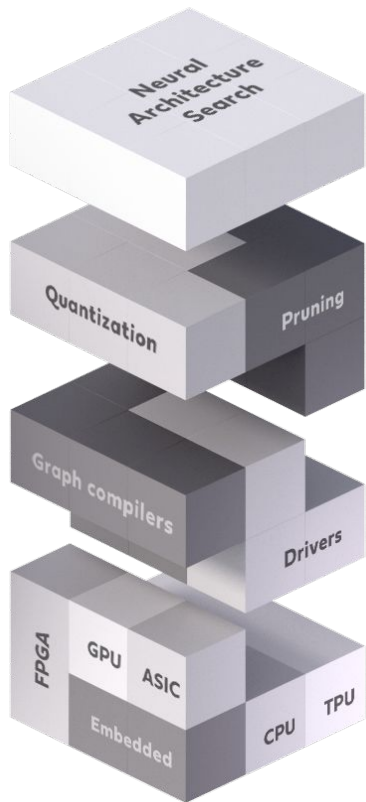
Neuromorphic computing - brain-inspired computing - has emerged as a new technology to enable information processing at very low energy cost using electronic devices that emulate the electrical behaviour of (biological) neural networks.

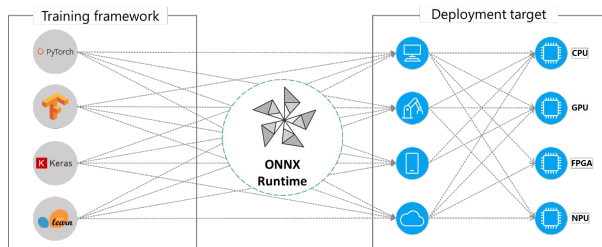
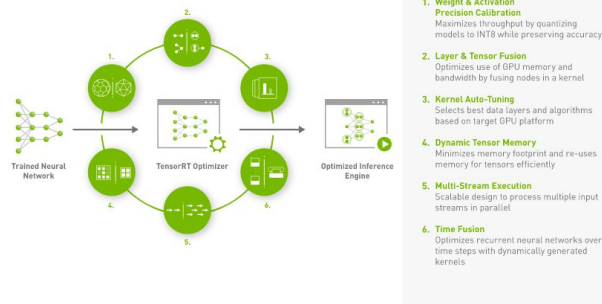
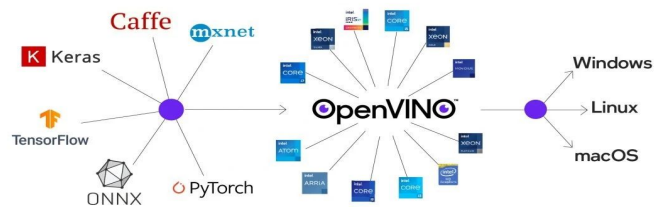
- Usually uses spiking neural networks (SNN)
- Neuromorphic chips attempt to model in silicon the massively parallel way the brain processes information as billions of neurons and trillions of synapses respond to sensory inputs such as visual and auditory stimuli.
- DARPA SyNAPSE program (Systems of Neuromorphic Adaptive Plastic Scalable Electronics) Intel Loihi 2, IBM TrueNorth, Tianjic; Stanford Neurogrid; HRL neuromorphic chip; Human Brain Project BrainScaleS-2, SpiNNaker and HICANN.

Other

- [DNA computing](#)
- [Unconventional computing: cellular automata, reservoir computing, using biological cells/neurons, chemical computation, membrane computing, slime mold computing and much more](#)

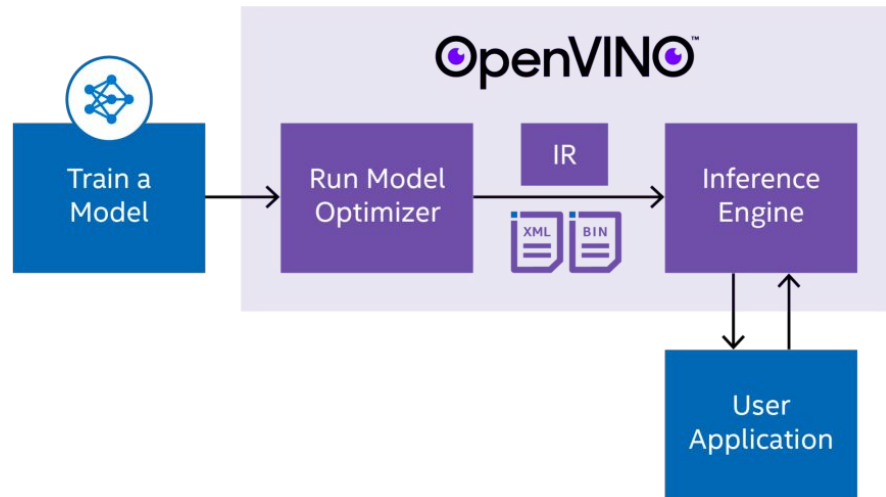
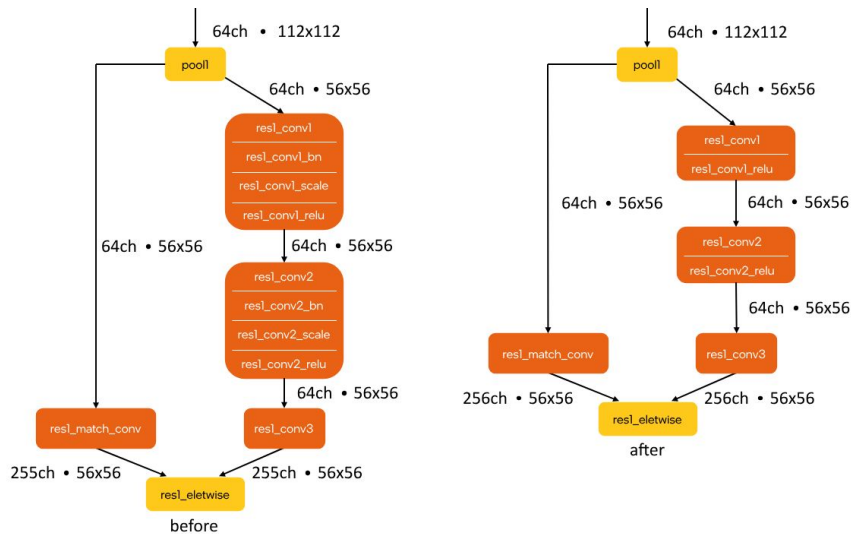
CPU	The central processing unit (CPU) is a general-purpose processor typically containing between 4 and 16 cores.
GPU	Graphics Processing Units (GPUs) are highly parallel cores (100 or 1000) for high-speed graphics rendering. They provide high-performance processing and typically take up more space and consume more power than CPUs. Because of their large number of small cores, GPUs are well suited for AI workloads, facilitating both neural network training and AI operation.
FPGA	Field Programmable Gate Array (FPGA), a technology that creates a chip with a set of logic elements, flip-flops, sometimes RAM and programmable electrical connections between them, consumes less power than CPUs and GPUs. Can be reprogrammed on site by engineers with programming experience.
ASICs	Application-specific integrated circuits (ASICs) are custom logic designed using manufacturer circuit libraries and offer the benefits of low power consumption, speed and small footprint. However, they are time-consuming to develop and more expensive than other options, so ASICs are recommended for products that will operate in very high volumes.
	ASICs Types: - Image processing units (VPUs), image and image processors, and coprocessors - Tensor processing units (TPUs), such as the first TPU developed by Google for its machine learning platform, TensorFlow. - Neural Compute Units (NCU), including from ARM
QPU	A quantum processing unit (QPU) is the central component of a quantum computer or quantum simulator. It is a physical or simulated processor that contains a series of interconnected qubits that can be manipulated to compute quantum algorithms. A QPU uses the behavior of particles such as electrons or photons to perform certain types of calculations much faster than the processors in modern computers. QPUs rely on behavior called superposition, the ability of a particle to be in many states at once, described in a relatively new branch of physics called quantum mechanics. In contrast, CPUs, GPUs, and DPUs apply the principles of classical physics to electrical currents. This is why modern systems are called classical computers.
Neuromorphic chips	Neuromorphic computing—brain-inspired computing—has emerged as a new technology that can process information at very low energy costs using electronic devices that mimic the electrical behavior of (biological) neural networks.
Photonic computing	Оптические вычисления или фотонные вычисления используют световые волны, создаваемые лазерами или некогерентными источниками, для обработки данных, хранения данных или передачи данных для вычислений. На протяжении десятилетий фотоны обещают обеспечить более высокую пропускную способность, чем электроны, используемые в обычных компьютерах (см. оптические волокна).



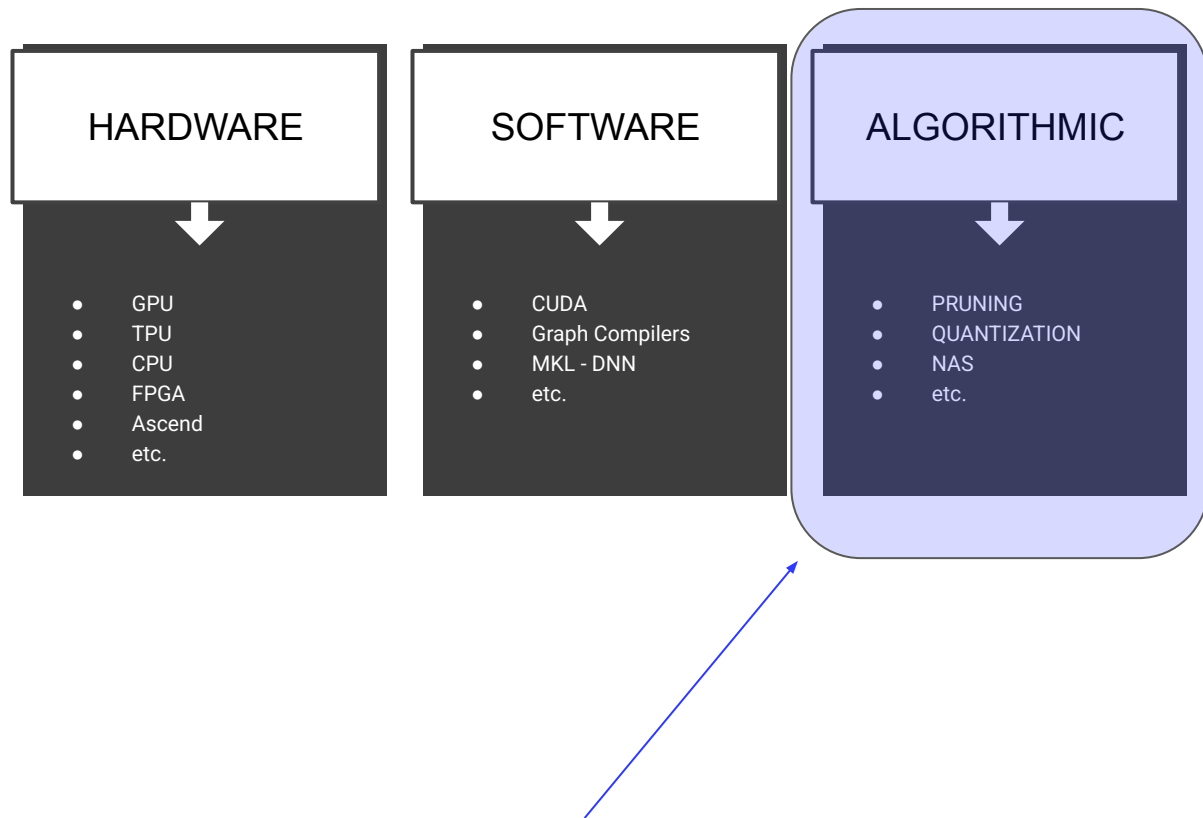
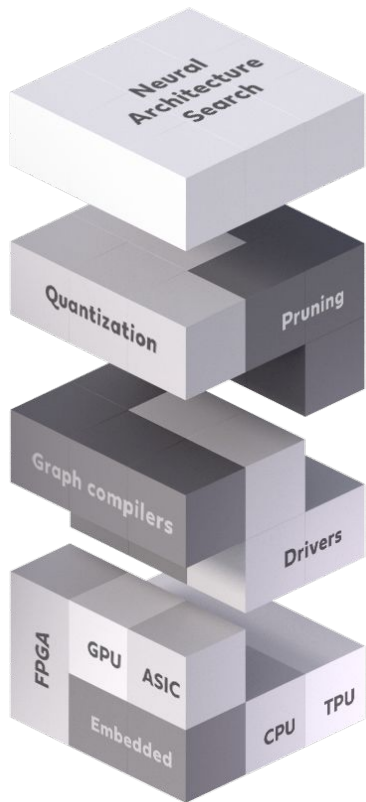


ONNX: This tool provides a standardized format for deep learning models, enabling them to be easily shared across different frameworks and platforms. In our project, ONNX used to ensure that our ResNet-50 model can be seamlessly transitioned between various environments.

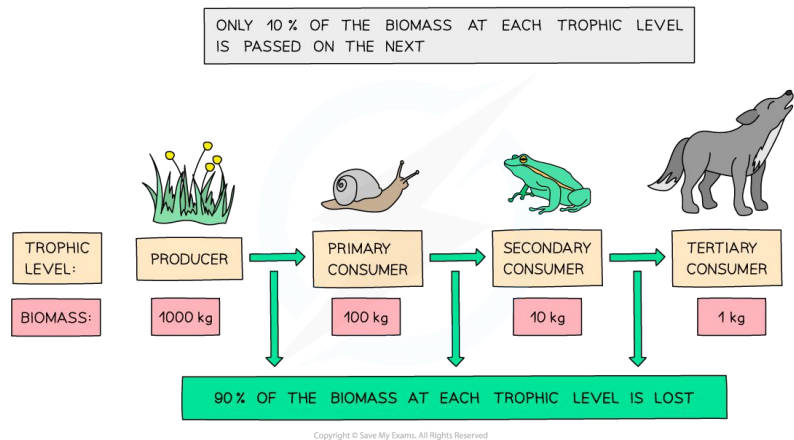
ONNX Runtime	Open source originally created by Microsoft and Facebook	Can be used as high level interface for TensorRT and OpenVino
TensorRT	Nvidia	This tool is specifically designed for NVIDIA GPUs, focusing on maximizing throughput and efficiency. It optimizes neural network models by fusing layers, selecting the most efficient data formats, and leveraging reduced precision (FP16) arithmetic where possible.
OpenVino	Intel	Developed by Intel, OpenVINO specializes in optimizing deep learning models for Intel hardware, particularly CPUs. It is a crucial tool for enhancing the performance of models where GPU resources are not available or limited.



To use quantization, some types of specification and various data formats we need specialized frameworks or we need to write low level solutions ourselves.



Adapted together



Brain consumes 20 joules energy per second



LLama-7B consumes 335 joules per request



Efficiency (Energy) loss at each level - each component was developed independently

Hardware

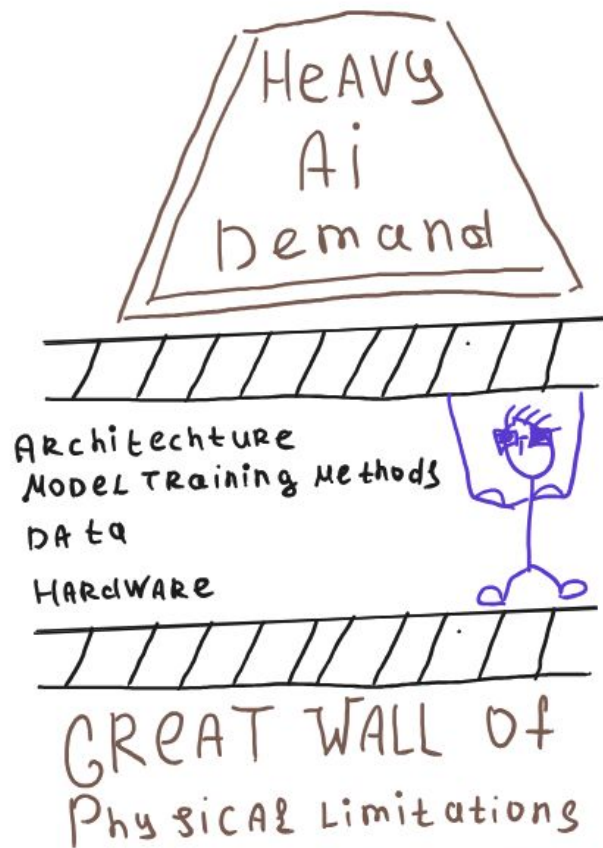
Software

Model

We covered some of the problems - let's recall them

What are possible solutions?

Summary



Questions?